Práctica 2: Introducción a Git y a GitHub

Laboratorio de Matemáticas

Grado en Ingeniería Matemática - Doble Grado en Física e Ingeniería Matemática

Miguel Beltra Vidal

Curso 2026/27

1. Objetivos

El objetivo de esta práctica es introducir el sistema de control de versión Git y la plataforma de desarrollo colaborativo GitHub. Al finalizarla, el estudiante será capaz de llevar un control de las diferentes versiones de sus proyectos y podrá almacenarlos en GitHub.

2. Sistemas de control de versiones

Por sistema de control de versiones se entiende un programa que es capaz de llevar un registro de las diferentes versiones de un proyecto, indicando qué cambios han sucedido: qué ficheros se han añadido, qué ficheros se han borrado o qué cambios se han producido en un fichero, entre otras cosas. Existen numerosos sistemas de control de versiones, como pueden ser Subversion, Mercurial o Perforce, pero en esta asignatura introduciremos Git, por ser el más popular y de código abierto. De acuerdo con la documentación oficial:

> *Git es un sistema de control de versiones distribuido, gratuito y de código abierto, diseñado para gestionar desde proyectos pequeños hasta proyectos de gran envergadura con rapidez y eficiencia.*

2.1. Plataformas de desarrollo colaborativo

Una plataforma de desarrollo colaborativo (también llamada Forja) es una plataforma que permite que albergar proyectos para que las personas puedan trabajar en ellos. GitHub es una plataforma de este tipo que permite alojar proyectos utilizando Git.

3. Primeros pasos con Git

3.1. Instalación de Git

3.1.1. Instalación en Linux

Para instalar Git en Linux, seguiremos el procedimiento habitual. Por ejemplo, si nuestra distribución es Ubuntu, entonces ejecutaremos los comandos siguientes en la consola:

```
$ sudo apt update
$ sudo apt install git
```

Comprobaremos que se ha instalado con el comando:

```
$ git --version
```

Nos debería dar información acerca de la versión de Git que hemos instalado.

3.1.2. Instalación en MacOS

Git viene incluido con las Xcode Command Line Tools, que son herramientas de desarrollo esenciales para macOS. Abre la Terminal (puedes buscarla en Spotlight con `Cmd + Espacio` y escribir `Terminal`). Ejecuta el siguiente comando para instalar las Xcode Command Line Tools:

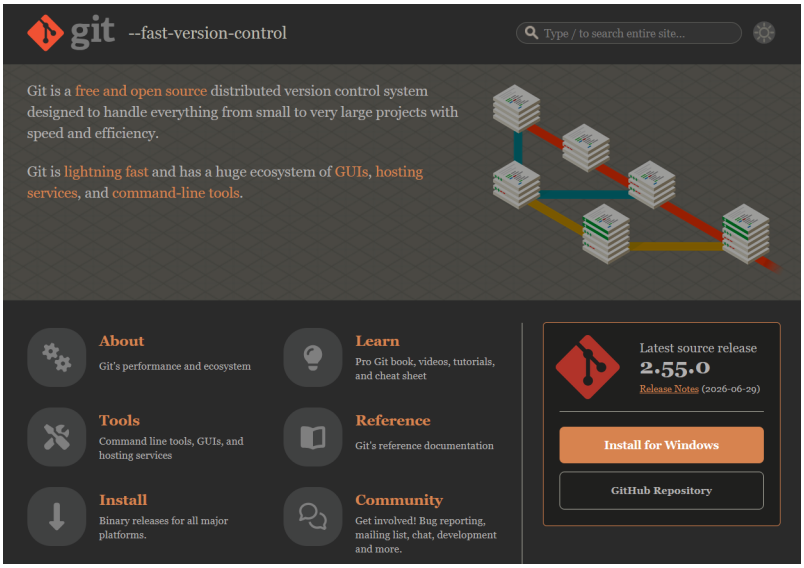
```
xcode-select --install
```

Una vez instalada, puedes ver que efectivamente se ha instalado con el comando siguiente:

```
git --version
```

3.1.3. Instalación en Windows

Entramos en la web oficial: `https://git-scm.com/`. Desde aquí podemos descargar el instalador. Para este curso será suficiente con marcar las opciones de configuración sugeridas por el instalador.



Podemos comprobar que se ha instalado correctamente desde la consola de comandos `cmd` de Windows ejecutando el comando:

```
C:\Users\user> git --version
```

3.2. Configuración de usuario en Git

Configuraremos nuestro usuario de Git con los siguientes comandos:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu.email@example.com"
```

3.3. Primer proyecto y primeros comandos

En la consola de comandos de nuestro sistema operativo nos situaremos en el directorio que contendrá el código de nuestras prácticas, y ejecutaremos el siguiente comando para iniciar nuestro repositorio de Git:

```
git init
```

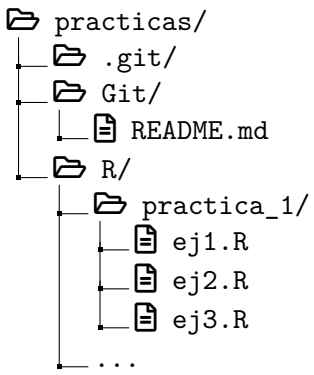
A continuación, crearemos un directorio que contendrá los ficheros con los que trabajaremos en nuestra práctica. Por ejemplo, añade a la estructura de directorios de la práctica 1 un nuevo directorio que se llame `Git`. Dentro de este, crearemos un fichero llamado `README.md`. Con tu editor de texto de preferencia, escribe en él algo como lo siguiente:

```
# Práctica de Git y GitHub

*Autor:* Miguel Beltrá Vidal
```

El fichero de texto que acabamos de crear es de tipo `Markdown`. Estos ficheros tienen la extensión `.md` y suelen utilizar para escribir ficheros de documentación, como pueden ser los habituales `README.md` que se encuentran en la mayoría de proyectos. En una práctica futura estudiaremos `Markdown`. Por el momento, es suficiente con imitar el estilo de los ejemplos de esta práctica.

La estructura de directorios debería quedar algo así:



Es posible que no veas el directorio `.git`. Esto se debe a que, como norma general, directorios y ficheros que empiezan por un punto (`.`) se interpretan como ficheros ocultos y el explorador de archivos en su configuración por defecto no los muestra. No pasa nada, ya que éste directorio contiene los ficheros necesarios para el control de versiones de Git y nosotros no necesitaremos editarlos. Una vez creado nuestro repositorio de Git y añadido el fichero `README.md`, ejecutaremos el siguiente comando en la terminal:

```
git status
```

Nuestra salida debería ser similar a la siguiente:

```
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Git/
        R/

nothing added to commit but untracked files present (use "git add" to
    ↪ track)
```

Como puedes observar, se resalta en color rojo como fichero sin seguimiento el fichero `README.md`. Esto quiere decir que ese fichero no forma parte de los ficheros cuyas versiones queremos controlar. Cuando subamos nuestros archivos a GitHub, cosa que explicaremos más adelante, estos ficheros no se subirán. Para añadirlo a la lista de ficheros a seguir, emplearemos el comando `git add [nombre_del_fichero]`. Añadiremos este fichero y comprobaremos el estado del repositorio de Git:

```
git add Git/README.md
git status
```

Nuestra salida debería ser similar a la siguiente:

```
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file: Git/README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        R/
```

Ahora ya sabemos que tenemos un nuevo archivo en seguimiento. Para confirmar los cambios, deberemos ejecutar el comando:

`git commit -m "Mensaje con la informacion de los cambios".`

En nuestro caso, podemos hacer lo siguiente:

```
git commit -m "Initial commit"
```

Tras ejecutar este comando, obtendremos la siguiente salida, confirmando los cambios:

```
[master (root-commit) ec3a5e4] Initial commit
 1 file changed, 6 insertions(+)
 create mode 100644 Git/README.md
```

Si ahora miramos el estado del repositorio, con `git status`, podemos ver que no existe ningún cambio que deba ser confirmado, aunque seguimos con ficheros sin seguimiento.

```
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        R/

nothing added to commit but untracked files present (use "git add" to
    ↪ track)
```

Realizaremos ahora un cambio en nuestro fichero `README.md`, añadiendo algo más de información. Por ejemplo:

```
# Práctica de Git y GitHub

*Autor:* Miguel Beltrá Vidal

## Descripción de la práctica

Este directorio lo utilizaremos para aprender los fundamentos de Git.
```

Si guardamos el fichero editado y ejecutamos:

```
git status
```

Nuestra salida debería ser similar a la siguiente:

```
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified: Git/README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        R/

no changes added to commit (use "git add" and/or "git commit -a")
```

Se nos informa que el fichero `README.md`, que está bajo seguimiento, ha sufrido alguna modificación. Lo añadimos al repositorio.

```
git add Git/README.md
git status
```

Y nos aparecerá que todos los cambios se han añadido:

```
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified: Git/README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        R/
```

Finalmente, confirmaremos la modificación utilizando de nuevo el comando `git commit`:

```
git commit -m "Update Git/README.md"
```

Obtendremos la siguiente confirmación de cambios:

```
[master 2dd4ae5] Update Git/README.md
 1 file changed, 8 insertions(+)
```

Ejercicio 1. Crea un fichero de R llamado `prueba.R` con el siguiente código (es posible que ya lo tengas de una clase anterior. Si ese es el caso, entonces puedes moverlo al directorio `Git`):

```
area_rectangulo <- function(base, altura) {
  base * altura
}

volumen_esfera <- function(radio) {
  4 / 3 * pi * radio^3
}

f <- function(x, y, z) {
  x^2 + x*y + y^2 + y*z + z^2 + x*z
}

area_rectangulo(3, 5)
volumen_esfera(2)
f(2,0,1)
```

Añadelo a Git y confirma su adición con un mensaje explicativo, como puede ser `"Add prueba.R"`. A continuación añade las siguientes líneas de código al final del fichero:

```
area_triangulo <- function(base, altura) {
  base * altura / 2
}

area_triangulo(4, 3)
```

Guárda los cambios, añádelos al repositorio de Git y confirma los cambios con un mensaje tipo `"Update prueba.R"`.

3.4. El fichero `.gitignore`

Hay veces que en nuestro directorio tenemos ficheros de los cuales no queremos hacer un control de versiones ni guardarlos. Suelen ser ficheros que contienen datos sensibles (contraseñas o información personal), datos de configuración (propios de cada ordenador) o compilados, ya que con los ficheros de código podremos obtener el compilado cuando lo necesitamos. El fichero `.gitignore` lista todos aquellos ficheros que queremos ignorar para no llevar ningún control de versiones sobre ellos. En este apartado trataremos de entender el fichero `.gitignore`.

Crearemos un fichero en nuestro directorio, que podemos llamar `fichero_a_ignorar.txt`. Lo editamos, escribiendo algo como:

```
Fichero que Git ignorará
```

Git verá dicho fichero. Si ejecutamos:

```
git status
```

Veremos el fichero que hemos añadido.

```
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Git/fichero_a_ignorar.txt
        R/

nothing added to commit but untracked files present (use "git add" to
    ↪ track)
```

Para evitar que Git tenga en cuenta el fichero, crearemos el fichero `.gitignore` en la raíz del repositorio (esto es, donde está el directorio `.git`), y en él escribiremos:

```
Git/fichero_a_ignorar.txt
```

Guardamos. Si volvemos a comprobar el estado:

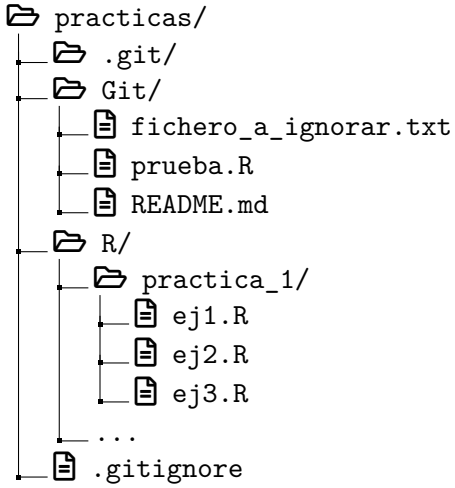
```
git status
```

Veremos el fichero que hemos añadido es ignorado por Git. Por el contrario, aparecerá el fichero `.gitignore` como fichero sin seguimiento.

```
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        R/

nothing added to commit but untracked files present (use "git add" to
    ↪ track)
```

La estructura de nuestro directorio de prácticas debería ser parecida a la siguiente:



Ejercicio 2. Añade el fichero `.gitignore` al seguimiento de versiones. Realiza una confirmación con un mensaje explicativo, p. ej. `"Add .gitinore file"`.

3.5. Añadir todos los ficheros con `git add .`

Recuerda que si modificamos el fichero `README.md` del directorio `Git` y lo queremos añadir al repositorio, podemos utilizar el comando `git add Git/README.md`. Si tuvieramos muchos ficheros, entonces tendríamos que ir uno por uno. Si queremos añadir todos los ficheros al mismo tiempo, entonces utilizamos:

```
git add .
```

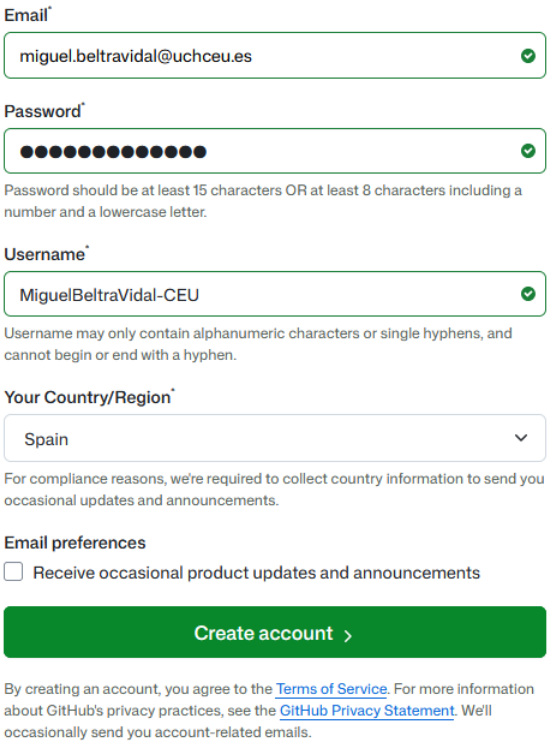
Es muy importante haber incluido en el `.gitignore` los ficheros de los que no queremos llevar un seguimiento. De lo contrario, al ejecutar el comando `git add .` incluiremos todos ellos también.

Ejercicio 3. Añade uno de los ficheros de la Práctica 1 al control de versiones, por ejemplo, el que tiene el ejercicio 1, y confirma añadiendo un mensaje explicativo. A continuación, añade todos los ficheros de la Práctica 1 que tengas hasta el momento. Ees posible que aun no hayas completado la Práctica 1 al momento de hacer este ejercicio. No pasa nada. A medida que vayas completando el trabajo, añade los cambios y confírmalos como hemos explicado en esta práctica.

4. Uso de GitHub

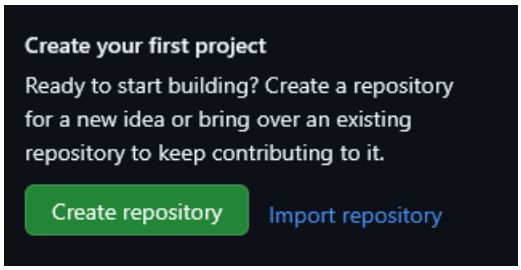
4.1. Registro en GitHub

Lo primero que deberemos hacer es registrarnos en la plataforma GitHub.

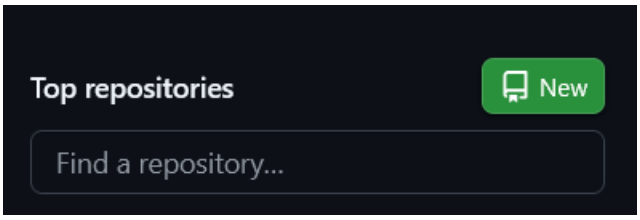


4.2. Crear un nuevo repositorio

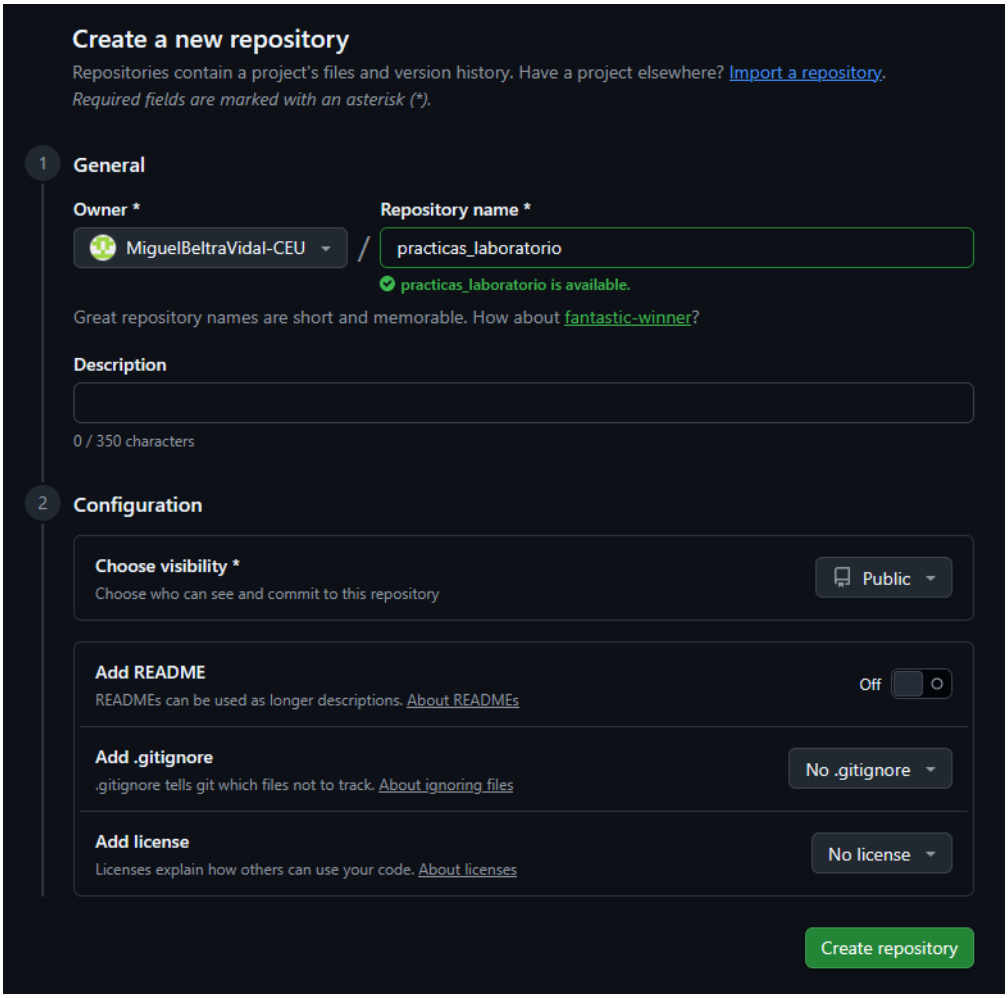
Desde la vista principal de GitHub podrás crear un nuevo repositorio con el botón `Create repository`:



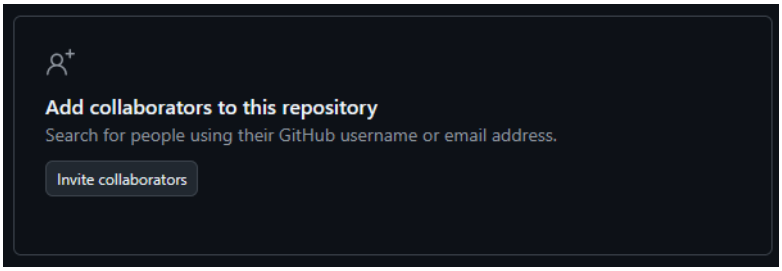
o clickando en `New` si ya tienes alguno creado:

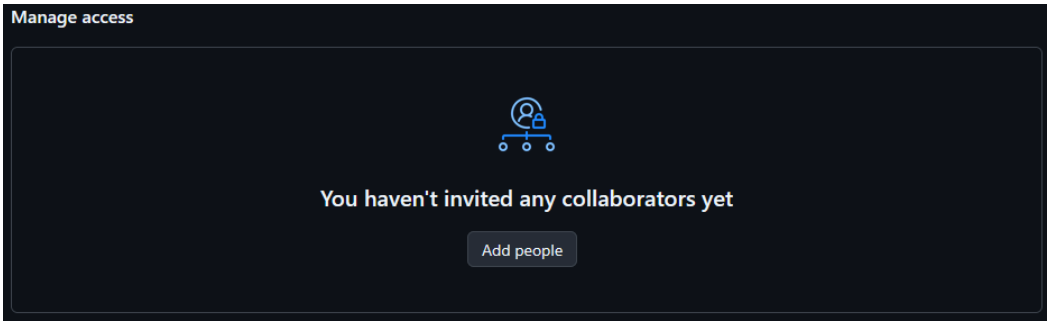


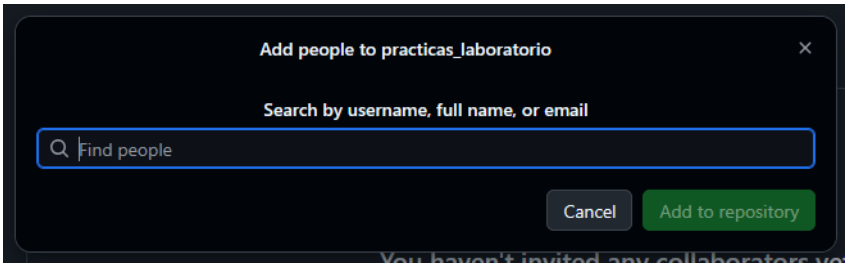
Pondremos un nombre al repositorio y lo marcaremos como privado.



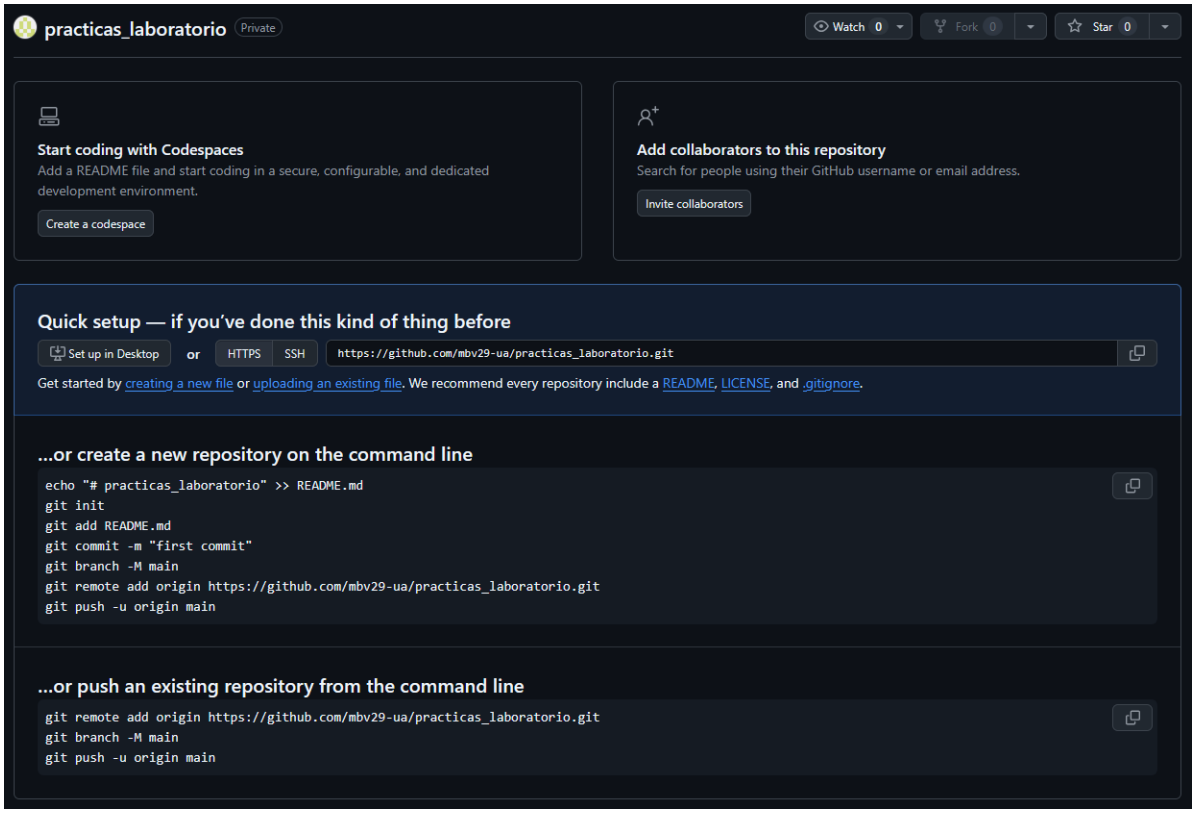
Invitaremos a un colaborador, en este caso, al profesor de la asignatura (`MiguelBeltraVidal-CEU`):







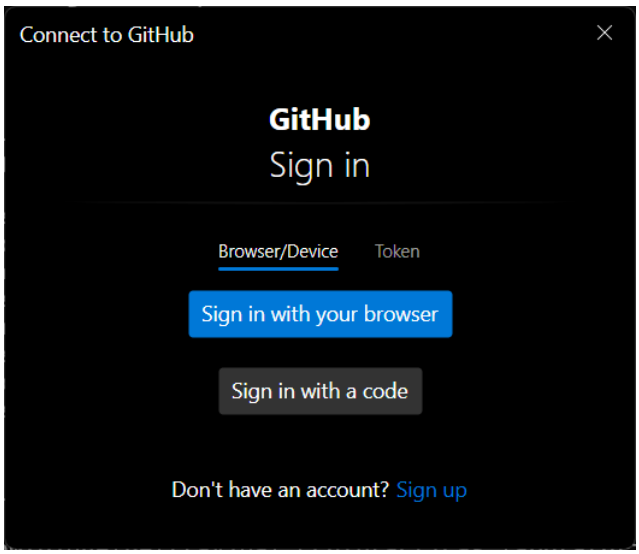
Podremos subir nuestro trabajo. Desde la vista principal de nuestro aparecen varias opciones para subir nuestro trabajo.

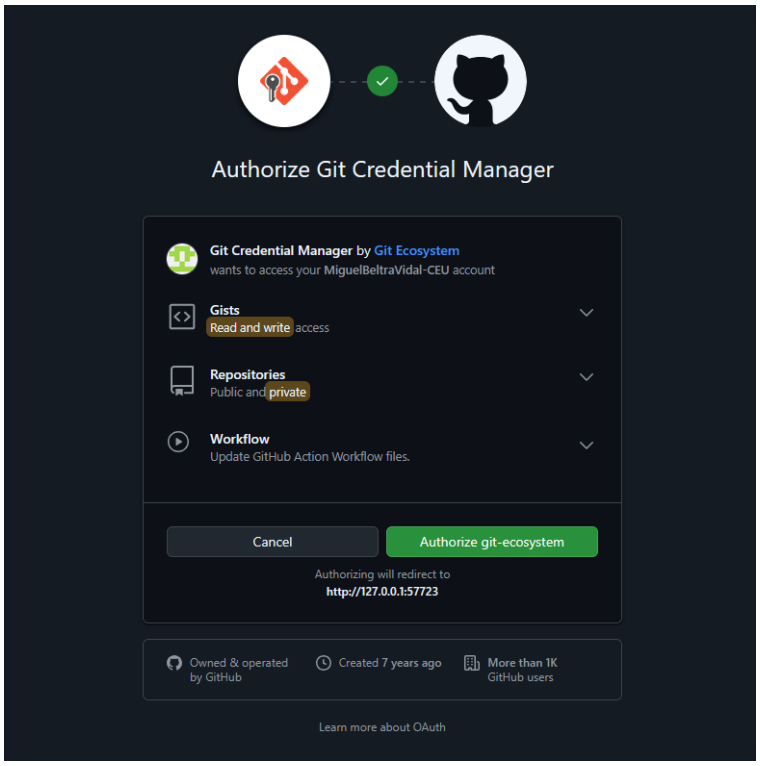


Como queremos subir un repositorio que ya existe en nuestro ordenador, nos fijaremos en la de abajo que tiene los comandos siguientes:

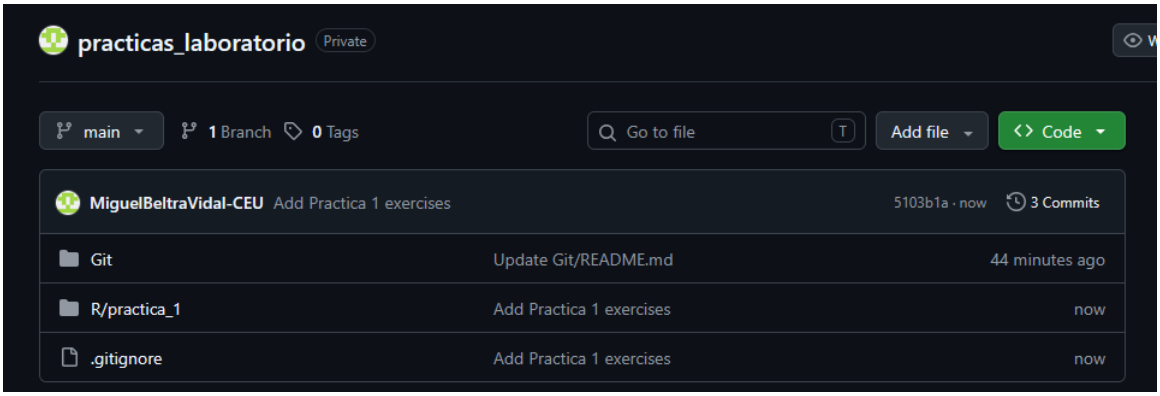
```
git remote add origin https://github.com/nombre_usuario/
    ↪ practicas_laboratorio.git
git branch -M main
git push -u origin main
```

Copiamos esos comandos desde GitHub y los ejecutamos en la terminal. El primer comando sirve para que nuestro repositorio local (el de nuestro ordenador) conozca el repositorio externo (el de GitHub). El segundo hace un renombrado de nuestra rama (de momento no trabajaremos con varias ramas). El último comando es el que permite subir los cambios a GitHub. Nos pedirá que autoricemos al gestor de credenciales, y ya lo tendremos.





Con esto, ya tendríamos nuestros ficheros subidos en GitHub.



5. Entrega y consejos

Asegúrate de invitar al profesor a tu repositorio de prácticas para que pueda ver que las estás haciendo. Asegúrate de que en GitHub aparezcan las últimas versiones de tus códigos. Deberás subir todos los ejercicios de la Práctica 1.

El flujo de trabajo habitual deberá ser el siguiente:

Antes de empezar

1. Si tu ordenador no tiene Git, instálalo y configura tu usuario.
2. Una vez tienes Git:
   a) Si en tu ordenador no tienes tus ficheros, clona tu repositorio de GitHub con

   ```
   git clone [direccion_del_repositorio]
   ```

   Desde la página de tu repositorio de GitHub puedes los detalles del `git clone` que vas a ejecutar.

   b) Si en tu ordenador tienes una versión antigua de los ficheros, traete de GitHub la última versión con el comando

   ```
   git pull
   ```

Durante tu sesión de trabajo

1. Cada poco tiempo, cada vez que hagas un cambio importante, añadelo al repositorio con

   ```
   git add [nombre_fichero]
   ```

   o, si quieres añadir todos los cambios, con

   ```
   git add .
   ```

2. Confirma los cambios con:

   ```
   git commit -m "Comentario explicativo de los cambios"
   ```

3. Súbelos a GitHub con:

   ```
   git push
   ```